ImgGrader v1.1 — User Guide

Automated grading of image-processing assignments

Contents

1 What ImgGrader does **1**

2 Quick start: grade one image **2**

3 The main window **2**

4 Rules **4**
- 4.1 Adding a rule . . . 4
- 4.2 Metrics . . . 5
- 4.3 How tolerances become points . . . 6
- 4.4 Notes keywords . . . 7
- 4.5 Console output rules . . . 7

5 Grading a whole class (batch) **8**
- 5.1 Folder layout . . . 8
- 5.2 How the files are found . . . 8
- 5.3 Running the batch . . . 9
- 5.4 The CSV file . . . 9

6 Masks and regions of interest **10**

7 Rule files **11**
- 7.1 Scan Assignment Folder . . . 11

8 Troubleshooting **12**

1 What ImgGrader does

ImgGrader compares the images (and, optionally, the console output) that students' programs produce with the instructor's reference results, and turns the comparison into points. You describe the grading as a list of *rules*. Each rule says which file to check, which measure to use (for example MSE or IoU), and how many points each result is worth. You can grade one submission on screen (**Run Single Eval**) or a whole class at once (**Batch Evaluate**); a batch writes one CSV file with every value and score.

Installation. Python 3.8 or newer is required. In the program folder run

```
pip install -r requirements.txt
python GraderApplicationCode.py
```

2 Quick start: grade one image

You need a reference image (for example `result.png`) and a student's version of it (for example `result_stud.png`).

1. Click **Load Expected Image** 1 and choose the reference image.
2. Click **Load Student Image** 2 and choose the student's image. Both images appear in the previews.
3. Click **Add Rule (popup)** 11. Choose the metric **MSE**, set **Max score** to 4 and **Tolerance(s)** to `1,4,25,100`, and click **Add**. (The Output field is already filled with the name of the student image.)
4. Click **Run Single Eval** 13. The **Result** column shows the measured value and the points, and a window lists all results (Figure 1).

Section 4.3 explains how the points are computed from the tolerances.

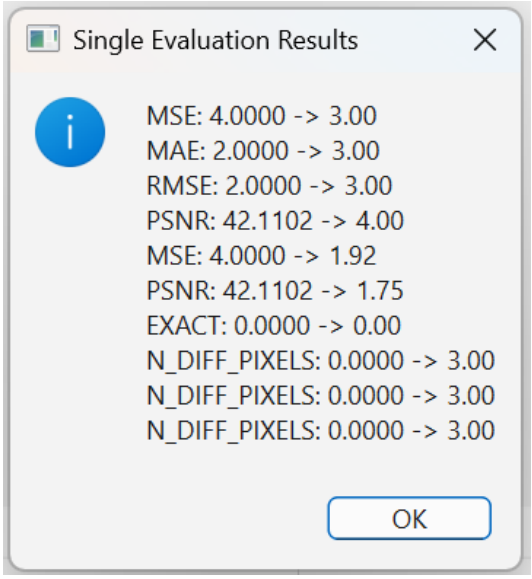


Figure 1: Result window of **Run Single Eval** with ten rules (the same values appear in the Result column). The student image is the reference with +2 added to every colour channel, so MSE = 4 (3 of 4 points), MAE = 2 and PSNR = 42.11 dB.

3 The main window

The window has five areas (Figure 2): **A** the toolbar, **B** the previews of the expected (left) and student (right) image, **C** the rules table, **D** the rule and grading buttons, and **E** the rules as JSON text with three utility buttons. The numbers 1–19 in Figures 3 and 4 are used throughout this guide.

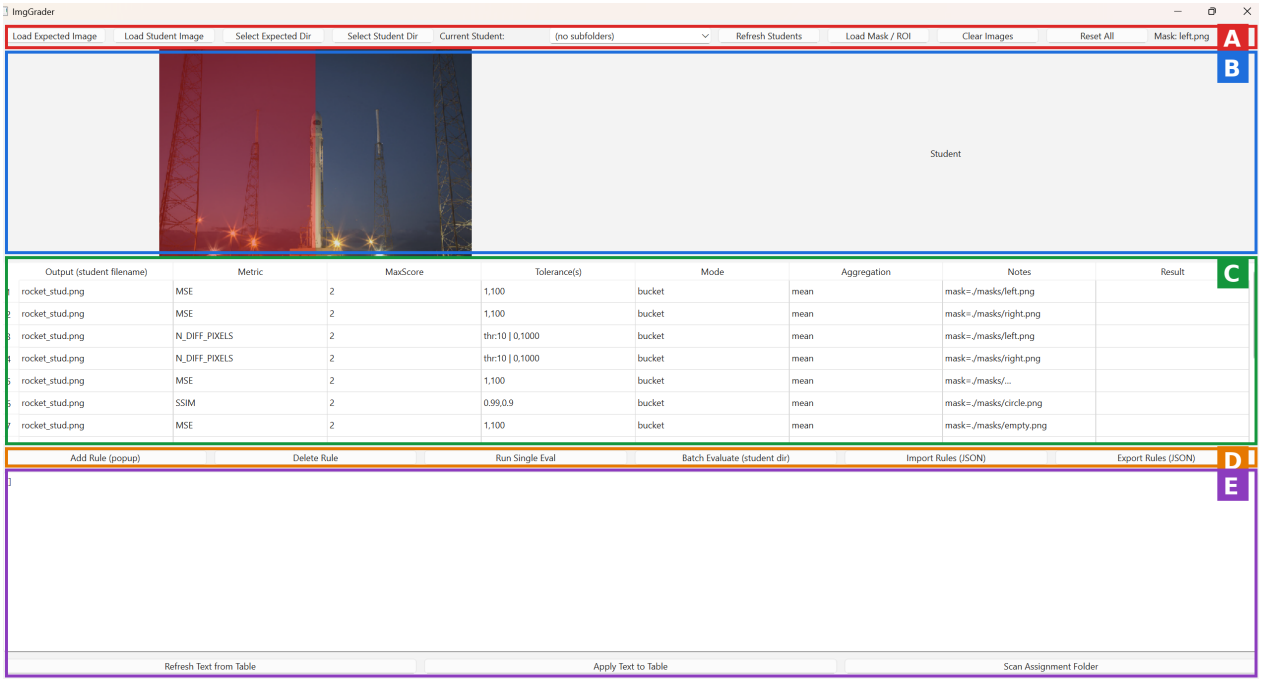


Figure 2: The main window. Here a mask (`left.png`) is loaded, so the expected image is shown with a red overlay on the masked area.

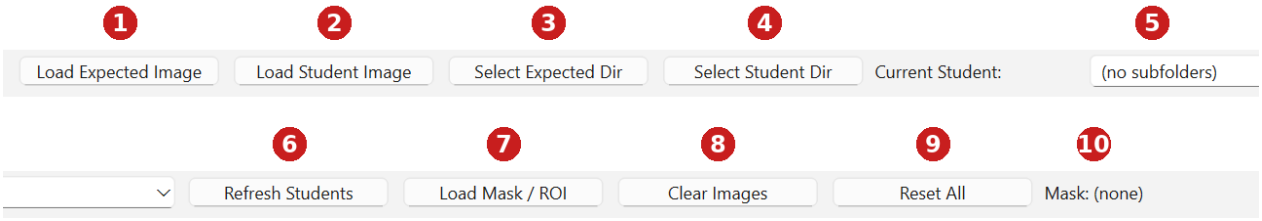


Figure 3: Toolbar (area A), shown in two halves.

	Control	What it does
1	Load Expected Image	Loads one reference image. Used by **Run Single Eval**; if it is still loaded during a batch, it is used as the reference for *every* rule (Section 5.2).
2	Load Student Image	Loads one student image for **Run Single Eval**.
3	Select Expected Dir	Folder with all reference files, for batch grading.
4	Select Student Dir	Folder that contains one subfolder per student.
5	Current Student	Lists the student subfolders. **Run Single Eval** reads console output (`cout` rules) from the student chosen here.
6	Refresh Students	Reads the Student Dir again, e.g. after a new submission folder was added.
7	Load Mask / ROI	Loads a mask file (**No**) or lets you draw a rectangle (**Yes**); see Section 6.
8	Clear Images	Removes the two loaded images; the rules stay.
9	Reset All	Clears images, rules table, rules text box, mask and folders.
10	Mask indicator	Name of the mask loaded with 7, or *(none)*.

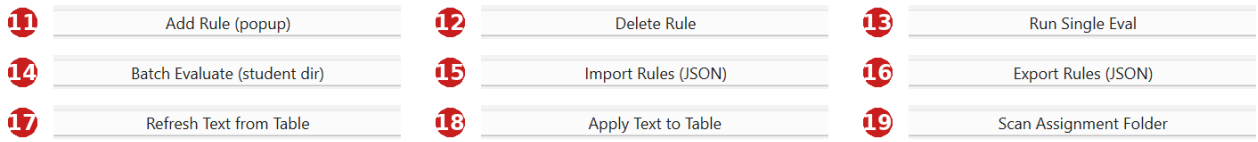


Figure 4: Rule and grading buttons (area D) and utility buttons (area E).

	Button	What it does
11	Add Rule (popup)	Opens the rule editor (Section 4.1).
12	Delete Rule	Deletes the selected row of the rules table.
13	Run Single Eval	Grades the two loaded images (and `cout` rules) with all rules.
14	Batch Evaluate (student dir)	Grades every student folder and saves a CSV file (Section 5).
15	Import Rules (JSON)	Loads a rule file; replaces the rules in the table.
16	Export Rules (JSON)	Saves the rules in the table to a rule file.
17	Refresh Text from Table	Shows the rules of the table as JSON text in area E.
18	Apply Text to Table	Replaces the table with the rules written in area E.
19	Scan Assignment Folder	Drafts rules from the code files of an assignment (Section 7.1).

4 Rules

Each row of the rules table is one rule. A student's total is the sum of the points of all rules.

Column	Meaning
Output (student filename)	The file the student's program must produce, e.g. `mask_stud.png`; or `cout` for console output. Left empty, the rule uses the same file as the rule above it (batch).
Metric	How the student file is compared with the reference (Table 4). For `cout`: `exact`, `contains` or `regex`.
MaxScore	Points for a perfect result (decimals such as 1.5 are allowed).
Tolerance(s)	Thresholds that turn the measured value into points (Section 4.3); for `cout` rules: the expected text.
Mode	`bucket` (points in steps) or `continuous` (points proportional to the value).
Aggregation	Only for MSE, MAE and ΔE: how the per-pixel errors are combined (`mean`, `median`, `max`, `trim10` = mean without the 10 % smallest and 10 % largest values).
Notes	Optional settings (Table 5), e.g. `mask=./masks/ring.png`.
Result	Filled by **Run Single Eval**: value and points, or an error message.

4.1 Adding a rule

Add Rule (popup) 11 opens the rule editor (Figure 5).

- **Output** is pre-filled with the name of the loaded student image; change it to the name students must use.

- Choosing a **Metric** fills in typical tolerances (last column of Table 4); adjust them to your assignment.
- **Aggregation** is shown only for MSE, MAE and ΔE.
- Typing `cout` as Output switches the metrics to `exact`/`contains`/`regex` and shows the field **Expected console output**.
- If a mask is loaded with 7, it is added to the new rule's Notes.

All cells of the table can also be edited directly by double-clicking them.

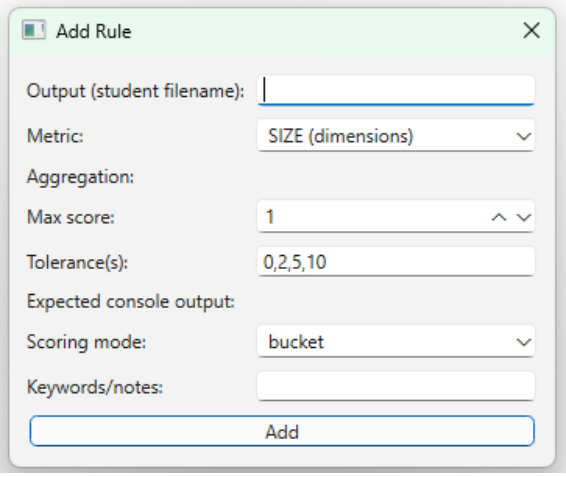


Figure 5: The rule editor.

4.2 Metrics

"Higher" means a larger value is better (1.0 or more is perfect); "lower" means a smaller value is better (0 is perfect). The direction decides how the tolerances are read (Section 4.3).

Table 4: Metrics. All image metrics compare the student image with the reference image.

Metric	**What it measures / when to use it**	**Better**	**Default tolerances**
EXACT	1 if every pixel is identical, else 0. Very strict. Without tolerances: full points or 0.	higher	`1`
MSE	Mean squared difference of the pixel values (0–255). Filtering, denoising.	lower	`1,5,10,100`
MAE	Mean absolute difference; less sensitive to a few very wrong pixels.	lower	`1,5,10,50`
RMSE	Square root of MSE, in grey levels.	lower	`1,2,5,10`
PSNR	$20 \log_{10}(255/\text{RMSE})$ in dB. Identical images give ∞ (full points).	higher	`30,25,20,15`
SSIM	Structural similarity of the grey images (blur, compression).	higher	`0.95,0.9,0.8`
NCC	Normalised cross-correlation of the grey images; ignores brightness and contrast changes, but negative for an inverted image.	higher	`0.95,0.9,0.8`
IoU	Overlap of two binary masks (pixels > 127 are foreground); segmentation.	higher	`0.95,0.9,0.8`

Metric	What it measures / when to use it	Better	Default tolerances
ΔE	Mean CIEDE2000 colour difference; colour transforms, white balance.	lower	`1,2,10,50`
N_DIFF_PIXELS	Number of pixels that differ by at least a threshold (default 10 grey levels), e.g. inside a ring around vessels.	lower	`thr:10 | 100,1000,5000`
Objects	Difference in the number of objects (connected white regions).	lower	`0`
Holes	Difference in the number of holes (black regions enclosed by white).	lower	`0`
Euler	Difference of the Euler numbers (objects − holes); catches a spurious hole that barely changes IoU.	lower	`0`
Contours	1 if both images have the same number of contours, else 0.	higher	`1`
ContoursStrict	Agreement of the number of holes per object (0–1).	higher	`1,0.75,0.5`
SIZE	Largest difference in width or height, in pixels. The student image is not resized for this rule.	lower	`0,2,5,10`

Tip

If a student image has a different size than the reference, it is resized to the reference size before comparison (except for SIZE). The CSV records this in the `resized` column, so you can see who submitted the wrong size — add a SIZE rule if the size itself should cost points.

4.3 How tolerances become points

Tolerances are numbers separated by commas. Their order does not matter; ImgGrader sorts them from best to worst.

Bucket mode. n thresholds create $n+1$ bands. The best band gives the full Max score, each following band one n-th less, and a value beyond the last threshold gives 0. For "lower is better" a value belongs to a band if it is *at most* the threshold, for "higher is better" if it is *at least* the threshold.

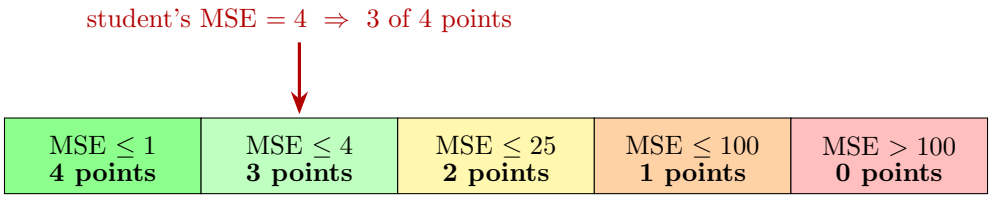


Figure 6: Bucket scoring of the rule "MSE, Max score 4, tolerances `1,4,25,100`" (lower is better).

Continuous mode. Points grow or shrink in proportion to the value; only the largest threshold $t_{\max}$ is used:

$$\text{higher is better:}\quad \text{points} = \text{Max score} \times \min\left(\frac{\text{value}}{t_{\max}},\, 1\right),$$

$$\text{lower is better:}\quad \text{points} = \text{Max score} \times \max\left(0,\, 1 - \frac{\text{value}}{t_{\max}}\right).$$

Example: MSE, Max score 2, tolerance `100`: a student with MSE = 4 gets $2 \times (1 - 4/100) = 1.92$ points. Points always stay between 0 and the Max score.

Special cases.

- **EXACT** without tolerances gives full points if the images are identical, otherwise 0.
- An **empty** tolerance (also "`-`" or "`---`") or one that is not a list of numbers (e.g. `0.9;0.8`) is reported as `ERROR: no tolerances set` or `ERROR: invalid tolerances` and gives 0 points — never full points by accident.
- **N_DIFF_PIXELS** takes the pixel threshold first: `thr:10 | 0,50,500` counts pixels that differ by 10 or more and gives full points for 0 such pixels, then steps at 50 and 500.

4.4 Notes keywords

Notes are `key=value` pairs, for example `gray=1; mask=./ring.png`. Separate several pairs with commas or semicolons.

Table 5: Keywords in the Notes column.

Keyword	Effect
`mask=path`	This rule uses only the pixels inside this mask (Section 6). A relative path is read from the folder of the imported rule file.
`gray=1`	N_DIFF_PIXELS: compare grey values instead of the three colour channels.
`logic=any` / `logic=all`	N_DIFF_PIXELS on colour images: a pixel counts if *any* channel differs by the threshold (default) or only if *all* channels do.
`thr=10`	N_DIFF_PIXELS: pixel threshold (alternative to `thr:` in the tolerances).
`file=name.txt`	`cout` rules: read the console output from this file instead of `stdout.txt`.

4.5 Console output rules

To grade text that a program prints, students save it as `stdout.txt` in their folder (or another file named with `file=`). Set Output to `cout`, choose the metric, and type the expected text in Tolerance(s):

- `exact`: the whole output equals the text. Spaces and empty lines at the start and end are ignored, and Windows or Unix line endings make no difference.
- `contains`: the text appears somewhere in the output.
- `regex`: the regular expression matches somewhere, e.g. `Area:\s*15\d\d`.

A match gives the full Max score, otherwise 0. A rule without expected text gives `ERROR: no expected text set`, because an empty text would match any output. In **Run Single Eval**, the output is read from the student selected in **Current Student** 5 (Figure 7).

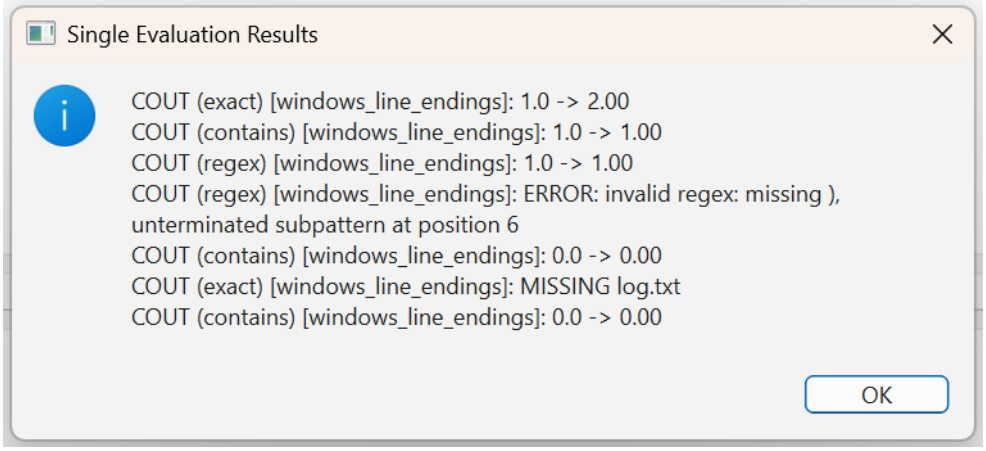


Figure 7: Seven console rules for one student: three matches, an invalid regular expression, two texts that do not appear, and a rule that reads a missing file `log.txt`.

5 Grading a whole class (batch)

5.1 Folder layout

Put the reference files in one folder and each student's files in their own subfolder:

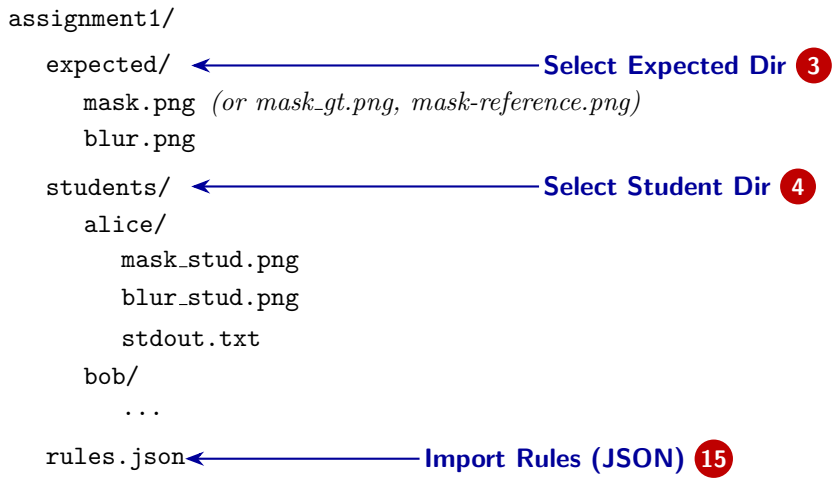


Figure 8: Folder layout for batch grading. Each subfolder of `students` is one student; its name becomes the student's name in the CSV.

5.2 How the files are found

Student file. ImgGrader searches the student's folder, including its subfolders, for the file named in Output (upper/lower case does not matter). If there is none, it accepts a file with the same name without the endings `_stud`, `_student`, `-stud` or `-student`, also with another image type (e.g. `.jpg` instead of `.png`). If two files qualify, the rule is reported as `AMBIGUOUS student` and gives 0 points; ImgGrader never guesses.

Reference file. If an image is loaded with **Load Expected Image** 1, it is the reference for *every* rule. Otherwise ImgGrader looks in the Expected Dir (including subfolders) for the student file name without its student ending, also with one of the reference endings `_gt`, `_ref`, `_reference` or `_expected` (or the same with `-` instead of `_`): for `mask_stud.png` it finds `mask.png`, `mask_gt.png` or `mask-reference.png`.

Watch out

An image left loaded with **Load Expected Image** from an earlier single evaluation is used for all rules of the batch. For an assignment with several reference files, click **Clear Images** 8 before **Batch Evaluate**.

5.3 Running the batch

1. **Reset All** 9, then **Import Rules (JSON)** 15 (or add rules).
2. **Select Expected Dir** 3 and **Select Student Dir** 4.
3. **Batch Evaluate (student dir)** 14, choose the name of the CSV file, and wait for the progress window to close.

A Student Dir without subfolders is graded as a single student.

5.4 The CSV file

The CSV has one row per student and, for every rule n with metric M, the following columns (in rule order), then `total_score`:

Column	Content
Rn_M_mask	Mask file used for this rule (empty: whole image).
Rn_M_student_path	Student file that was graded, or why none was found.
Rn_M_expected_path	Reference file that was used.
Rn_M_resized	1 if the student image had another size and was resized.
Rn_M_val	Measured value, or a status such as `MISSING student` (Table 7).
Rn_M_score	Points for this rule.
total_score	Sum of all points.

Example (two rules of one student, path columns left out):

student	R1_MSE_val	R1_MSE_score	R5_MSE_val	R5_MSE_score
offset_plus2	4.0	3.0	4.0	1.92

Rule 1 is the bucket rule of Figure 6, rule 5 the continuous example of Section 4.3. A PSNR value of `inf` means the images are identical.

Table 7: Statuses in a `_val` column. All of them give 0 points.

Status	Meaning
`MISSING student`	No file with the expected name in the student's folder.
`AMBIGUOUS student`	More than one file qualifies; the student path column lists them.
`UNREADABLE student`	The file exists but is not a readable image (e.g. empty or damaged).
`MISSING expected` / `AMBIGUOUS expected`	No reference file, or more than one, in the Expected Dir.
`MISSING stdout.txt`	A `cout` rule found no output file.
`ERROR: ...`	The rule could not be evaluated; the message says why (Section 8).

6 Masks and regions of interest

A mask is a black-and-white image: white pixels (grey value above 127) are graded, black pixels are ignored. A mask of another size is scaled to the reference image. With a mask, every metric uses only the masked pixels; Objects, Holes, Euler and the contour metrics treat everything outside the mask as black. SIZE ignores masks.

One mask for all rules. Click **Load Mask / ROI** 7 and answer **No** to choose a mask file. The indicator 10 shows its name and the expected image shows the masked area in red (Figure 9). The mask is written into the Notes of every image rule in the table that has no mask of its own, so it is also saved when you export the rules.

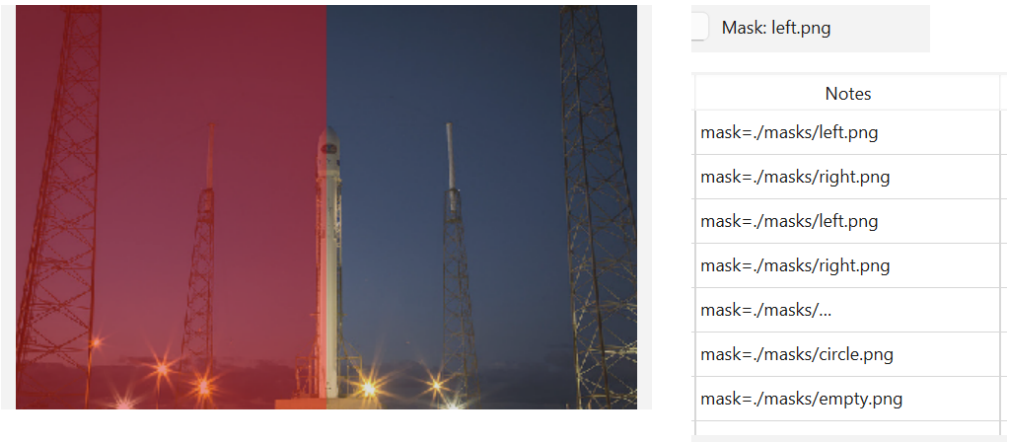


Figure 9: A global mask (`left.png`) is loaded. Left: the expected image with the masked area in red. Right: the mask indicator and the Notes column — rules that already have their own `mask=` keep it.

Drawing a rectangle. Answer **Yes** instead, drag a rectangle on the expected image, click **Use ROI**, and save the mask (Figure 10). The saved file has the size of the full image, with the rectangle in white.

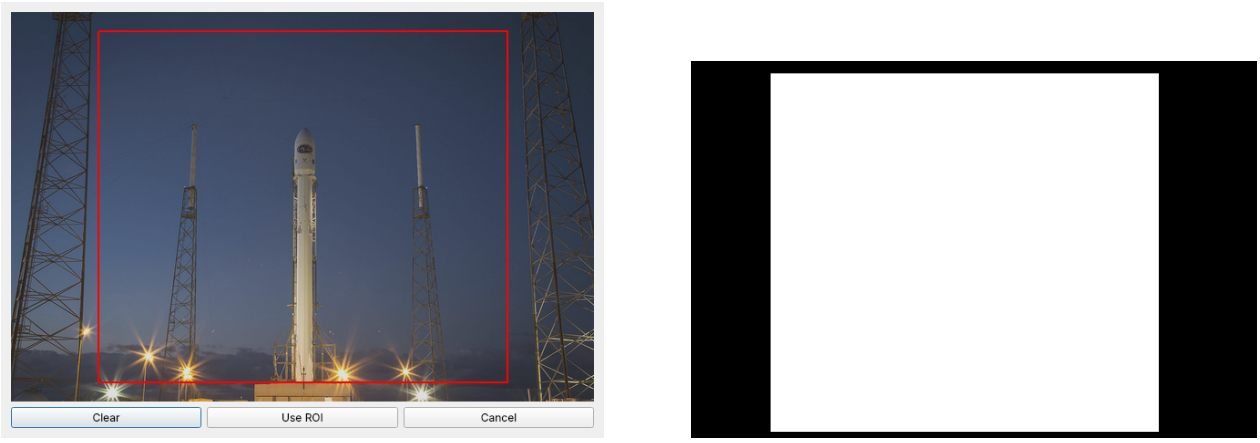


Figure 10: Left: drawing a rectangle. Right: a mask file saved this way (white = graded, black = ignored).

A mask per rule. Write `mask=path` in a rule's Notes, e.g. a ring around the vessels for an N_DIFF_PIXELS rule and the field of view for an IoU rule. A rule's own mask always wins over the global one.

Watch out

A mask that does not exist, cannot be read, or contains no white pixel is an error (0 points), never silently ignored (Figure 11).

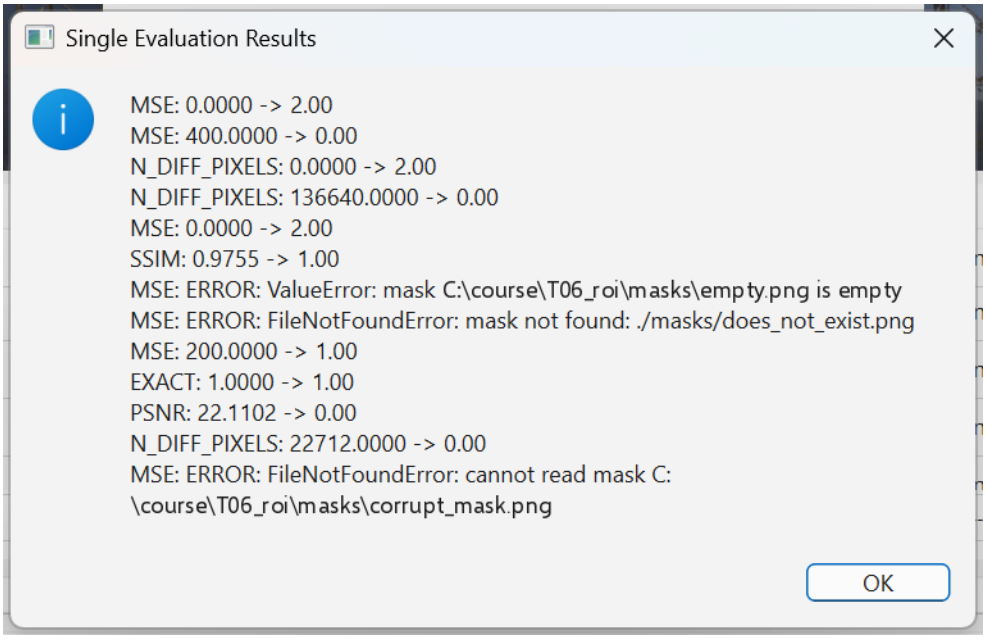


Figure 11: Thirteen rules with different masks on one student whose right image half is 20 grey levels too bright: MSE is 0 on the left half, 400 on the right half and 200 on the whole image. Rules with an empty, missing or damaged mask show ERROR.

7 Rule files

Export Rules (JSON) 16 saves the table as a rule file and **Import Rules (JSON)** 15 loads one, so the same rubric can be reused in every semester. One rule in a rule file looks like this:

```
{ "output": "mask_stud.png", "metric": "IoU", "max_score": 3,
  "tolerances": "0.95,0.9,0.8", "mode": "bucket",
  "aggregation": "mean", "notes": "mask=./masks/fov.png" }
```

Refresh Text from Table 17 shows all rules as such text in area E; after editing the text, **Apply Text to Table** 18 rebuilds the table. Text that is not valid JSON is rejected with a warning and the table stays unchanged (Figure 12).

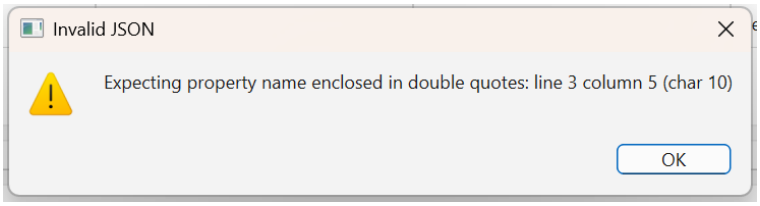


Figure 12: Warning after **Apply Text to Table** with invalid text; it tells where the error is, and the table keeps its rules.

7.1 Scan Assignment Folder

Scan Assignment Folder 19 reads the code files (`.py`, `.ipynb`, `.c`, `.cpp`, `.h`, `.hpp`, `.java`, `.txt`) of a folder you choose and drafts rules:

- an EXACT rule for every saved image whose name contains `_stud` or `_student` (`.png`, `.jpg`, `.jpeg`);
- a `cout` rule if the code prints text (`print(`, `std::cout`, `cout <<`).

The drafts are a starting point: choose the metrics you want, set tolerances, and type the expected text of the `cout` rule — until then it shows `ERROR: no expected text set` (Figure 13).

	Output (student filename)	Metric	MaxScore		Notes	Result
1	blur_stud.png	EXACT	1		from scan	No images loaded
2	cout	exact	1		file=stdout.txt	ERROR: no expected text set

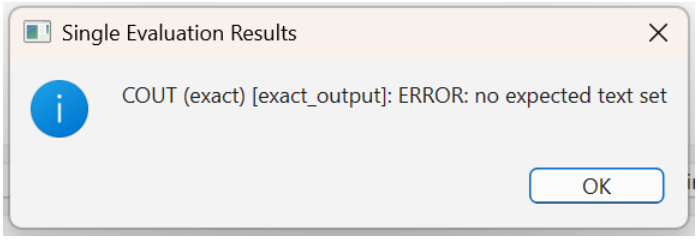


Figure 13: Top: rules drafted by Scan Assignment Folder (middle columns left out). Bottom: **Run Single Eval** shows that the `cout` rule still needs its expected text.

8 Troubleshooting

Message	What to do
Load both expected and student image to run single evaluation.	Load both images (1, 2). Rules with Output `cout` need no images.
No images loaded (Result column)	An image rule was run without loaded images; load them, or ignore it if you only test `cout` rules.
No student dir/image for cout	Select the Student Dir 4 and the student in 5.
`MISSING student`	Check the file name in Output against the student's file; see Section 5.2.
`AMBIGUOUS student`	The student's folder has two matching files; remove one or make Output more specific.
`UNREADABLE student`	The student's file is empty or damaged; the student must resubmit it.
`MISSING expected`	No reference with a matching name in the Expected Dir.
`ERROR: no tolerances set` / `ERROR: invalid tolerances`	Enter thresholds as numbers separated by commas (Section 4.3).
`ERROR: KeyError: "unknown metric"`	The metric name is misspelled; use a name from Table 4.
`ERROR: FileNotFoundError: mask not found`	Check the `mask=` path; a relative path is read from the folder of the rule file.
`ERROR: ValueError: mask ... is empty`	The mask has no white pixel.
`ERROR: no expected text set`	Type the expected text of the `cout` rule in Tolerance(s).
`ERROR: invalid regex`	Correct the regular expression (e.g. a missing parenthesis).
`MISSING stdout.txt`	The student's folder has no output file (or set `file=` to the right name).

Message	What to do
Invalid JSON	Correct the text in the text box, or click **Refresh Text from Table** to start again.
All rules use the same reference in a batch	An image is still loaded with 1; click **Clear Images** 8.